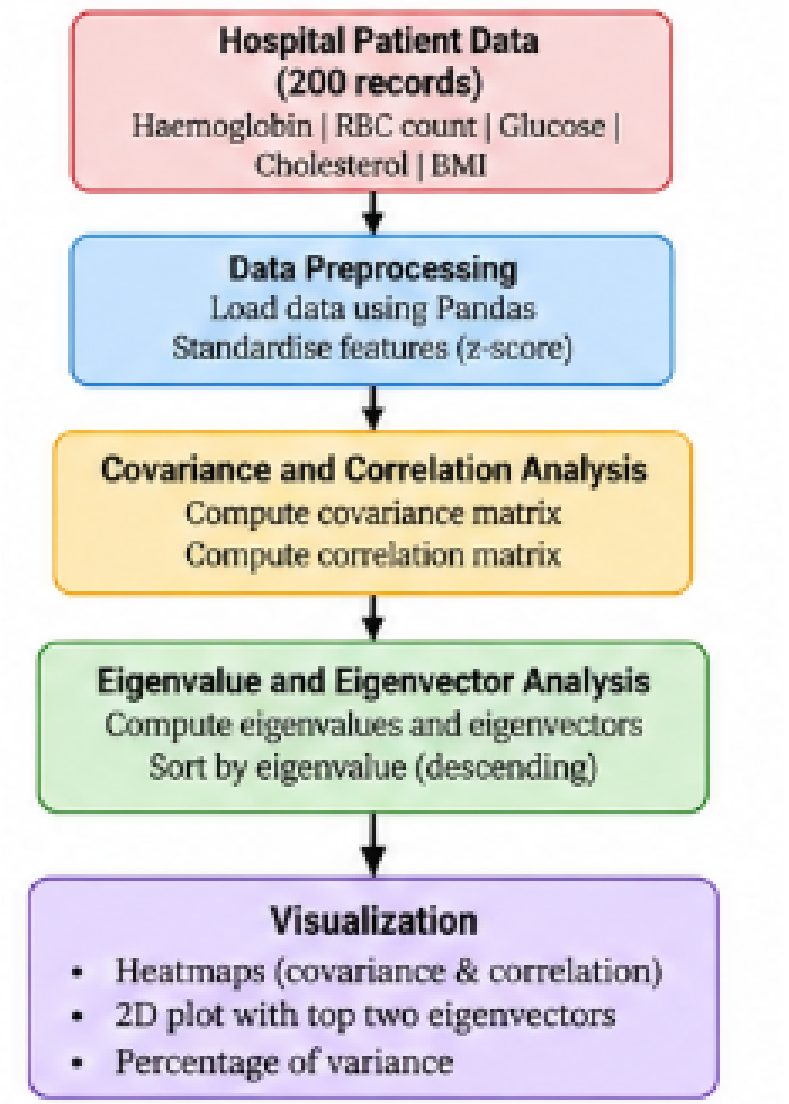
Experiment 2: Covariance, Eigenvalues and Eigenvectors Analysis

1. AIM

To analyse the relationships between medical parameters such as haemoglobin, RBC count, glucose, cholesterol, and BMI using covariance and correlation matrices. To calculate eigenvalues and eigenvectors using NumPy, identify the principal directions of variation, and visualize the results using heatmaps and a 2D plot.

SYSTEM ARCHITECTURE



2. MATHEMATICAL FORMULAS

1. Standardization (Z-score)

$$\boxed{Z = \frac{X - \mu}{\sigma}}$$

2. Covariance

$$\boxed{\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^{n} (X_i - \bar{X})(Y_i - \bar{Y})}$$

3. Correlation Coefficient

$$\boxed{r_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}}$$

$$-1 \leq r_{XY} \leq 1$$

4. Eigenvalues and Eigenvectors

$$\boxed{Av = \lambda v}$$

Alternatively,

$$\boxed{\det(A - \lambda I) = 0}$$

5. Percentage of Variance Explained

$$\boxed{\text{Variance Explained (\%)} = \frac{\lambda_i}{\sum_{j=1}^{p} \lambda_j} \times 100}$$

6. Cumulative Variance Explained by the First Two Principal Components

$$\boxed{\text{Cumulative Variance (\%)} = \frac{\lambda_1 + \lambda_2}{\sum_{j=1}^{p} \lambda_j} \times 100}$$

7. Covariance Matrix

$$\boxed{C = \frac{1}{n-1} X^T X}$$

CODE

```
# Experiment 2: Covariance, Eigenvalues and Eigenvectors Analysis

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Generate a sample dataset of 200 patient records
np.random.seed(42)

n = 200

haemoglobin = np.random.normal(13.5, 1.5, n)
rbc = 3.5 + 0.1 * haemoglobin + np.random.normal(0, 0.3, n)
glucose = np.random.normal(110, 20, n)
cholesterol = 150 + 0.8 * glucose + np.random.normal(0, 15, n)
bmi = 18 + 0.08 * glucose + np.random.normal(0, 3, n)

df = pd.DataFrame({
    'Haemoglobin': haemoglobin,
    'RBC': rbc,
    'Glucose': glucose,
    'Cholesterol': cholesterol,
    'BMI': bmi
})

print("Original Dataset (First 5 Rows):")
print(df.head())

# 2. Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

df_scaled = pd.DataFrame(
    X_scaled,
    columns=df.columns
)
```

```
print("\nStandardized Dataset (First 5 Rows):")
print(df_scaled.head().round(3))

# 3. Calculate covariance and correlation matrices
cov_matrix = np.cov(X_scaled, rowvar=False)
corr_matrix = np.corrcoef(X_scaled, rowvar=False)

cov_df = pd.DataFrame(
    cov_matrix,
    index=df.columns,
    columns=df.columns
)

corr_df = pd.DataFrame(
    corr_matrix,
    index=df.columns,
    columns=df.columns
)

print("\nCovariance Matrix:")
print(cov_df.round(3))

print("\nCorrelation Matrix:")
print(corr_df.round(3))

# 4. Plot covariance and correlation heatmaps
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
sns.heatmap(
    cov_df,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)
plt.title("Covariance Matrix")

plt.subplot(1, 2, 2)
sns.heatmap(
    corr_df,
    annot=True,
    cmap='coolwarm',
    vmin=-1,
```

```
    vmax=1,
    fmt='.2f'
)
plt.title("Correlation Matrix")

plt.tight_layout()
plt.show()

# 5. Calculate eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

# Sort in descending order
indices = np.argsort(eigenvalues)[::-1]

eigenvalues = eigenvalues[indices]
eigenvectors = eigenvectors[:, indices]

print("\nEigenvalues:")
print(np.round(eigenvalues, 4))

print("\nEigenvectors:")
print(np.round(eigenvectors, 4))

# 6. Calculate percentage of variance explained
variance_percent = (
    eigenvalues / np.sum(eigenvalues)
) * 100

results = pd.DataFrame({
    'Principal Component': [
        f'PC{i+1}' for i in range(len(eigenvalues))
    ],
    'Eigenvalue': eigenvalues,
    'Variance Explained (%)': variance_percent
})

print("\nVariance Explained:")
print(results.round(3).to_string(index=False))

print(
    "\nCumulative variance explained by first two components:",
    round(np.sum(variance_percent[:2]), 2), "%"
)
```

```
# 7. Transform the data into two dimensions using PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    alpha=0.7
)

# Draw arrows representing the first two principal directions
for i in range(2):
    plt.arrow(
        0, 0,
        pca.components_[i, 0] * 3,
        pca.components_[i, 1] * 3,
        head_width=0.15,
        length_includes_head=True,
        label=f'PC{i+1}'
    )

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("2D PCA Visualization")
plt.grid(True)
plt.legend()
plt.show()
```

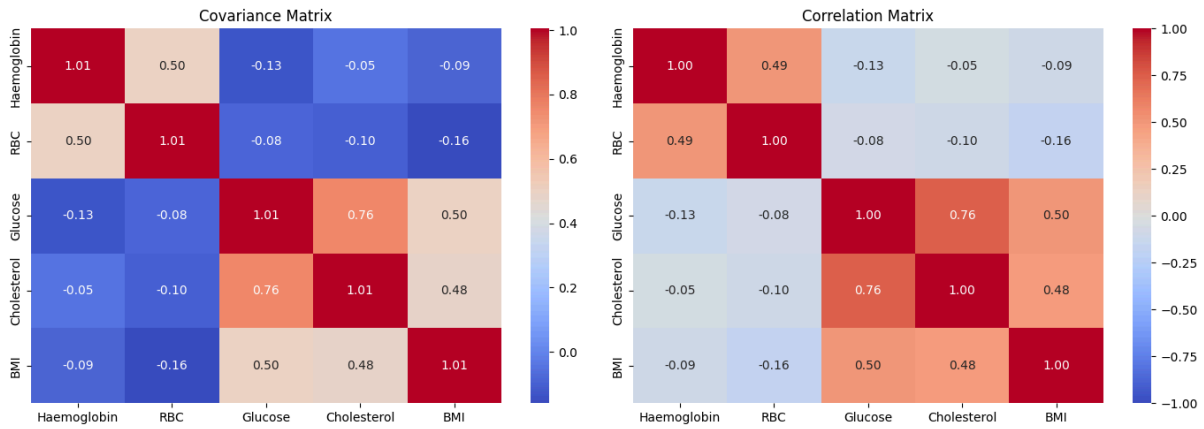
OUTPUT

```
Original Dataset (First 5 Rows):
   Haemoglobin       RBC     Glucose  Cholesterol        BMI
0    14.245071  5.031843   78.111447   223.843987  27.063767
1    13.292604  4.997496   98.012500   214.577520  24.292866
2    14.471533  5.272069  110.104874   251.127988  27.096752
3    15.784545  5.394595  110.939612   259.086257  25.488343
4    13.148770  4.401576  100.998691   237.000476  24.776407

Standardized Dataset (First 5 Rows):
   Haemoglobin    RBC  Glucose  Cholesterol    BMI
0        0.579  0.479   -1.522       -0.558  0.005
1       -0.105  0.378   -0.518       -0.959 -0.833
2        0.741  1.189    0.092        0.621  0.015
3        1.684  1.552    0.134        0.964 -0.472
4       -0.208 -1.383   -0.368        0.010 -0.687

Covariance Matrix:
             Haemoglobin    RBC  Glucose  Cholesterol    BMI
Haemoglobin        1.005  0.497   -0.135       -0.049 -0.092
RBC                0.497  1.005   -0.084       -0.098 -0.159
Glucose           -0.135 -0.084    1.005        0.759  0.505
Cholesterol       -0.049 -0.098    0.759        1.005  0.478
BMI               -0.092 -0.159    0.505        0.478  1.005

Correlation Matrix:
             Haemoglobin    RBC  Glucose  Cholesterol    BMI
Haemoglobin        1.000  0.495   -0.134       -0.049 -0.091
RBC                0.495  1.000   -0.084       -0.098 -0.159
Glucose           -0.134 -0.084    1.000        0.755  0.502
Cholesterol       -0.049 -0.098    0.755        1.000  0.475
BMI               -0.091 -0.159    0.502        0.475  1.000
```



```
Eigenvalues:
[2.2584 1.4254 0.6066 0.5004 0.2343]

Eigenvectors:
[[-0.2123  0.6741  0.3221 -0.6116 -0.1506]
 [-0.2305  0.6649 -0.2359  0.6599  0.1169]
 [ 0.5803  0.1956 -0.3418  0.0097 -0.7128]
 [ 0.5663  0.2342 -0.3149 -0.2699  0.6726]
 [ 0.4943  0.1017  0.7903  0.3429  0.0561]]

Variance Explained:
Principal Component  Eigenvalue  Variance Explained (%)
                PC1       2.258                  44.942
                PC2       1.425                  28.366
                PC3       0.607                  12.071
                PC4       0.500                   9.958
                PC5       0.234                   4.663

Cumulative variance explained by first two components: 73.31 %
```

